МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №4

Дисциплина: ТЕХНОЛОГИИ СОЗДАНИЯ И ПОДДЕРЖКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Тема: «Диаграммы компонентов и диаграммы последовательности»

Индивидуальное задание: сервис Flowra – персональный планировщик задач.

Работу выполнила: _____ Баннова.Д.А

Направление подготовки: 09.03.03 Прикладная информатика

Преподаватель: профессор кафедры анализа данных и искусственного интеллекта факультета компьютерных технологий и прикладной математики, к. техн. н., доцент

_____ Р.Ю. Вишняков

Краснодар

2026

Цель работы:

Изучение и получение практических навыков создания диаграммы компонентов и диаграммы последовательности в нотации UML на примере разрабатываемой интеллектуальной системы планирования задач Flowra.

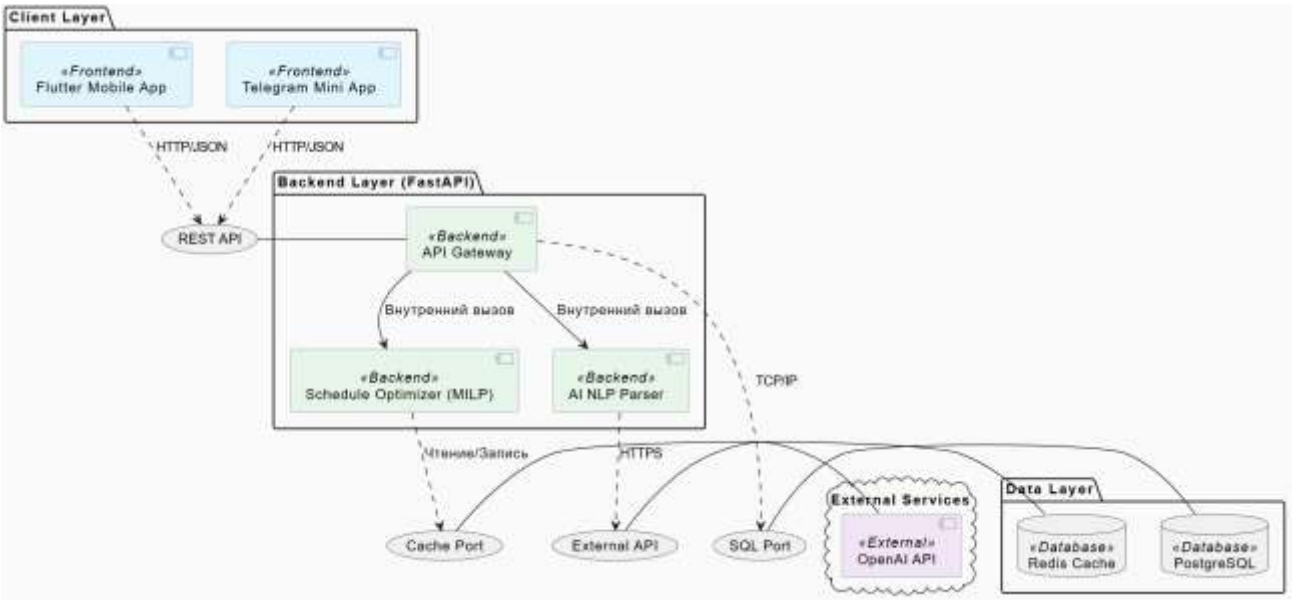
Введение:

В предыдущих лабораторных работах были определены требования к системе Flowra, описаны пользовательские сценарии (Use Cases) и спроектирована общая структурная архитектура. Данная лабораторная работа направлена на более глубокое проектирование системы. Диаграмма компонентов позволит разбить программный код приложения на физические модули (библиотеки, пакеты, исполняемые файлы) и показать зависимости между ними через интерфейсы. Диаграммы последовательности (Sequence Diagrams) позволят визуализировать динамику системы: как именно объекты и компоненты Flowra обмениваются сообщениями во времени при выполнении ключевых пользовательских задач (генерация плана и активация Rescue Mode).

ПРАКТИЧЕСКАЯ ЧАСТЬ

1. Диаграмма компонентов (Component Diagram)

В данном разделе представлена физическая структура системы Flowra.



Проект разделен на три основных логических слоя: клиентский, серверный и слой данных. Для связи компонентов используются предоставляемые и требуемые интерфейсы (порты).

Диаграмма компонентов отражает физическое разбиение системы на модули и их взаимодействие через программные интерфейсы. Архитектура построена по многослойному принципу:

1. Логические слои (Пакеты)

- **Client Layer (Уровень клиента):** содержит модули, с которыми пользователь взаимодействует напрямую (фронтенд).
- **Backend Layer (Уровень логики):** «сердце» системы на базе FastAPI, где происходят основные вычисления и маршрутизация данных.
- **Data Layer (Уровень данных):** отвечает за надежное хранение информации и быстрый доступ к ней.
- **External Services (Внешние сервисы):** облачные решения, расширяющие возможности системы (в данном случае — искусственный интеллект).

2. Описание компонентов

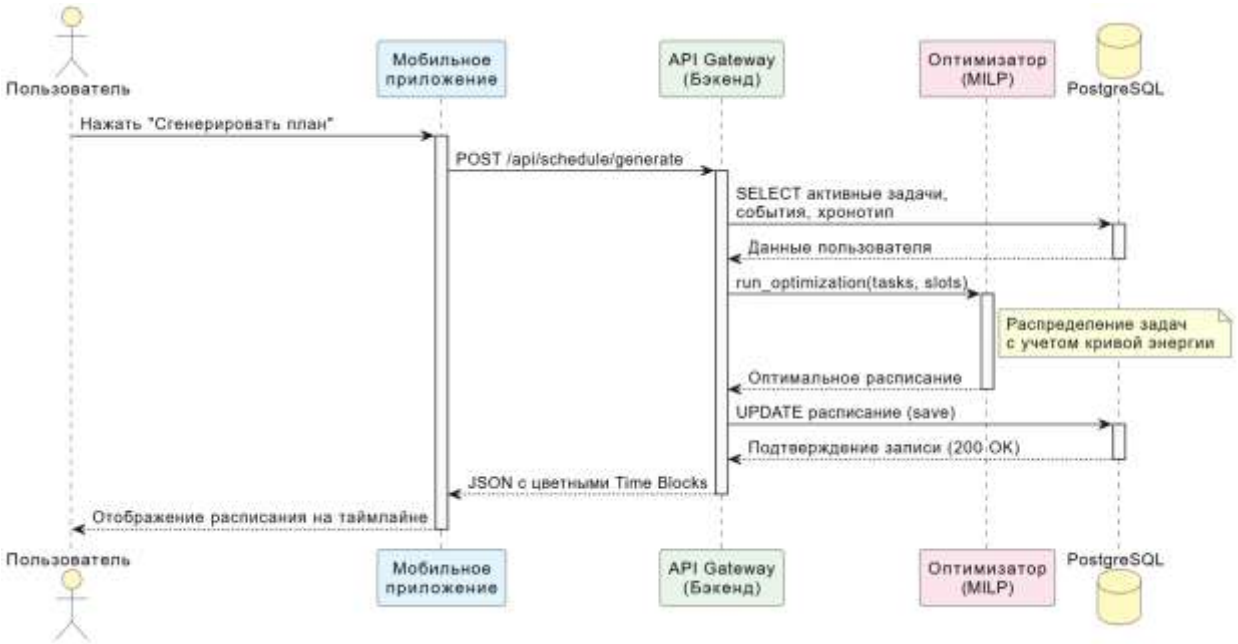
- **Flutter Mobile App:** Основное мобильное приложение. Оно отвечает за визуализацию таймлайна, прием настроек хронотипа и отображение цветных временных блоков.
- **Telegram Mini App:** Облегченная версия интерфейса внутри мессенджера Telegram, позволяющая пользователю быстро добавлять задачи «на ходу» без открытия основного приложения.
- **API Gateway:** Центральный диспетчер бэкенда. Он принимает запросы от клиентов, проверяет авторизацию пользователя и решает, какому модулю передать задачу дальше.
- **Schedule Optimizer (MILP):** Модуль, использующий методы линейного программирования для создания идеального расписания. Он берет список задач и «втискивает» их в свободные окна пользователя, учитывая приоритеты и кривую энергии.

- **AI NLP Parser:** Специализированный модуль для обработки текста. Если пользователь вводит задачу голосом или свободной фразой, этот модуль вычленяет из нее суть (название, длительность, дедлайн).
- **PostgreSQL:** Реляционная база данных. Здесь постоянно хранятся профили пользователей, их хронотипы, история выполненных задач и все настройки.
- **Redis Cache:** Сверхбыстрая база данных в оперативной памяти. Используется для временного хранения («кэширования») уже сгенерированных планов и очередей задач, чтобы система работала мгновенно.
- **OpenAI API:** Внешняя нейросеть, к которой обращается модуль NLP для качественного распознавания смысла человеческой речи.

3. Интерфейсы и связи

- **Интерфейс REST API:** Используется для связи клиентов с сервером. Фронтенд отправляет данные в формате JSON (например, новую задачу), а сервер возвращает ответ.
- **Внутренние вызовы (внутри Backend Layer):** API Gateway напрямую обращается к Оптимизатору и NLP-парсеру. Это быстрые вызовы внутри одной серверной среды.
- **External API (HTTPS):** Защищенное соединение с серверами OpenAI. Система передает туда «сырой» текст пользователя и получает обратно структурированные данные.
- **SQL Port (TCP/IP):** Стандартный канал связи бэкенда с базой данных PostgreSQL для выполнения запросов на чтение и запись.
- **Cache Port:** Высокоскоростной канал связи с Redis. Позволяет Оптимизатору мгновенно записывать промежуточные результаты расчетов, не нагружая основную базу данных.

2. Диаграмма последовательности: Сформировать план задач



Данная диаграмма визуализирует логику работы основного алгоритма системы Flowra (прецедент ВИ-03). Она описывает процесс взаимодействия между пользователем, интерфейсом и серверными модулями в хронологическом порядке (сверху вниз).

1. Участники процесса (Participants)

- **Пользователь (User):** инициатор процесса, взаимодействующий с мобильным интерфейсом.
- **Мобильное приложение (App):** клиентская часть (Frontend), которая принимает ввод и отображает финальный результат.
- **API Gateway (API):** центральный серверный узел (Backend), координирующий работу всех остальных модулей.
- **Оптимизатор (MILP):** математический модуль, который выполняет сложные расчеты по распределению задач.
- **PostgreSQL (DB):** база данных, обеспечивающая хранение исходной информации и сохранение результатов.

2. Пошаговый разбор сценария (Messages)

1. **Инициация:** Пользователь нажимает кнопку «Сгенерировать план». Мобильное приложение активируется и отправляет HTTP-запрос POST /api/schedule/generate на сервер.

2. **Сбор данных:** Бэкенд (API Gateway) не может строить план «в пустоте». Он обращается к базе данных PostgreSQL с SQL-запросом (SELECT), чтобы получить:
    - Список всех активных (невыполненных) задач пользователя.
    - Список «железных» событий (пары, встречи), которые нельзя двигать.
    - Данные о хронотипе (настройки «кривой энергии»).
3. **Передача в Оптимизатор:** Получив данные, Бэкенд вызывает метод run_optimization. В этот момент управление переходит к математическому модулю MILP.
4. **Математический расчет (Примечание на схеме):** Внутри Оптимизатора происходит самое важное: алгоритм распределяет задачи по свободным слотам времени. Он старается поставить самые сложные («глубокие») задачи на те часы, где кривая энергии пользователя максимальна.
5. **Возврат результата:** Оптимизатор возвращает Бэкенду структуру данных, которая представляет собой готовое «идеальное» расписание.
6. **Сохранение:** Чтобы план не пропал при закрытии приложения, Бэкенд отправляет команду UPDATE в базу данных. Система фиксирует новое расписание в памяти.
7. **Ответ клиенту:** Бэкенд отправляет мобильному приложению ответ в формате JSON. В нем содержатся координаты «цветных временных блоков» (Time Blocks) — время начала, конца и цвет каждой задачи.
8. **Финализация:** Мобильное приложение отрисовывает блоки на экранном таймлайне. Пользователь видит готовый план.

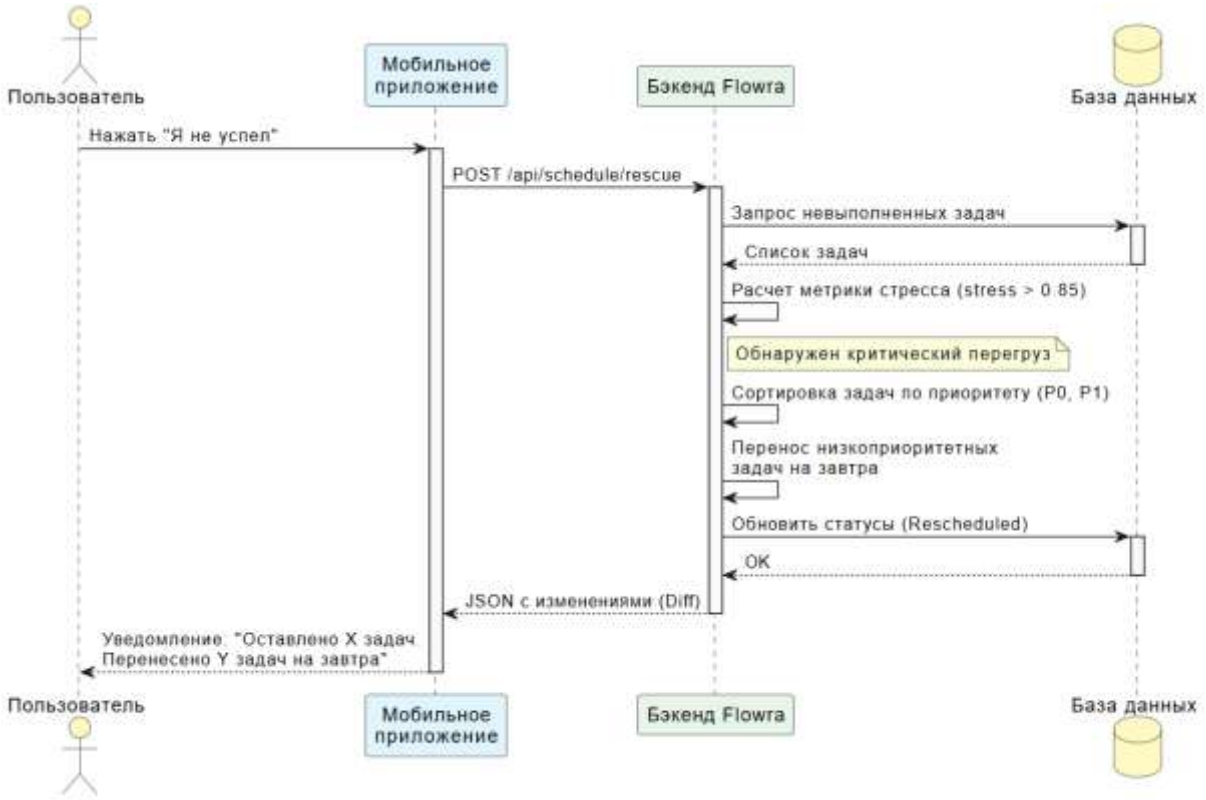
3. Технические особенности схемы

- **Фокус управления (Vertical bars):** Прямоугольники на линиях жизни показывают, какой компонент активен в данный момент. Видно, что пока Оптимизатор считает, Бэкенд ждет его ответа.

- **Синхронные вызовы:** Сплошные стрелки обозначают прямые запросы, а пунктирные — возвраты ответов. Это подчеркивает классическую клиент-серверную архитектуру приложения.

3. Диаграмма последовательности: Активация Rescue Mode (Овербукинг)

Вторая диаграмма визуализирует обработку исключительной ситуации — критического перегруза пользователя, когда алгоритм не может вместить все задачи в доступное время.



Подробное описание диаграммы последовательности: «Активация Rescue Mode (Овербукинг)»

Данная диаграмма визуализирует логику работы системы в ситуации критической нехватки времени (прецедент ВИ-09). Она описывает интеллектуальный процесс пересчета расписания с целью снижения нагрузки на пользователя.

1. Участники процесса (Participants)

- **Пользователь (User):** сообщает системе о том, что текущий план невыполним.

- **Мобильное приложение (App):** интерфейс, отображающий предупреждения и итоговый отчет о переносе задач.
- **Бэкенд Flowra (API):** основной вычислительный узел, принимающий решение о том, какие задачи оставить, а какие — перенести.
- **База данных (DB):** хранилище текущих статусов задач и обновленных планов.

2. Пошаговый разбор сценария (Messages)

1. **Сигнал о перегрузке:** Пользователь нажимает в приложении кнопку «Я не успел». Это происходит, когда в текущем дне осталось слишком много незавершенных задач. Приложение отправляет запрос POST /api/schedule/rescue.
2. **Загрузка контекста:** Бэкенд запрашивает из базы данных список всех задач, которые на данный момент имеют статус «не выполнено» или «в процессе».
3. **Анализ стресса (Internal Loop):** Бэкенд выполняет внутреннюю операцию — расчет метрики стресса.
    - *Логика:* Если объем оставшихся задач превышает доступное время до конца дня более чем на 85% (stress > 0.85), система подтверждает состояние овербукинга.
4. **Приоритизация (Internal Loop):** Бэкенд не просто двигает все задачи, а анализирует их важность:
    - Сначала выделяются задачи с высшим приоритетом (**P0** и **P1**). Они остаются в плане на сегодня.
    - Задачи с низким приоритетом автоматически помечаются для переноса.
5. **Перепланирование:** Система принимает решение о переносе низкоприоритетных дел на завтрашний день, очищая расписание пользователя на текущий вечер.
6. **Синхронизация:** Бэкенд отправляет в базу данных команду на обновление статусов задач (статус меняется на Rescheduled).

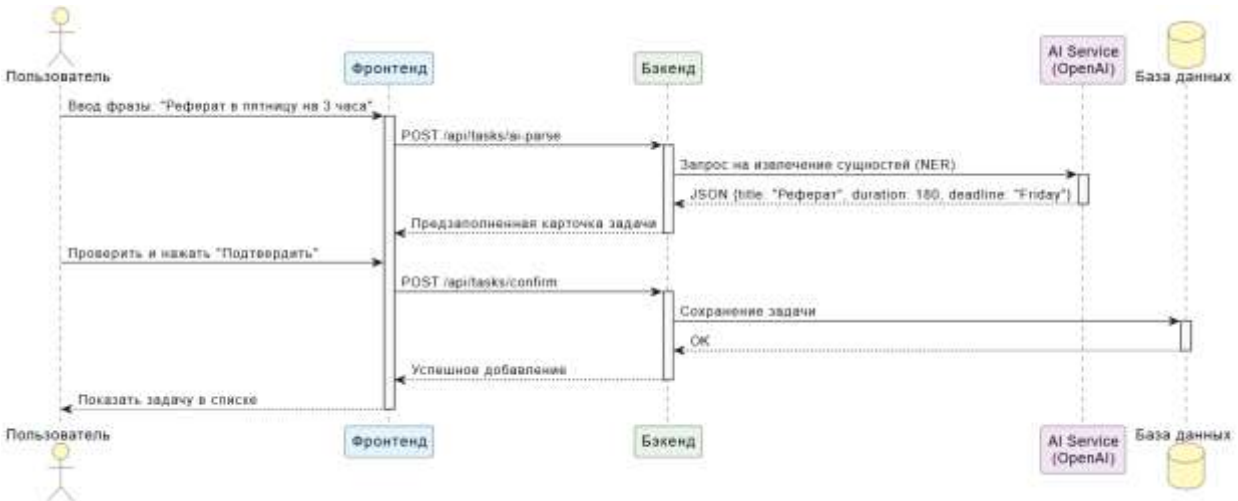
7. **Формирование отчета (Diff):** Вместо того чтобы просто обновить экран, Бэкенд отправляет приложению специальный объект «Diff» (разницу), в котором указано, что именно изменилось.
8. **Обратная связь:** Мобильное приложение выводит понятное уведомление: «Стресс снижен. Оставлено X важных задач, остальные (Y) перенесены на завтра». Это психологически разгружает пользователя.

3. Технические особенности схемы

- **Self-Message (Петли на Бэкенде):** Загнутые стрелки, направленные на сам Бэкенд, показывают внутренние алгоритмические проверки, которые не требуют обращения к базе данных или пользователю.
- **Note (Заметка):** На схеме выделено условие stress > 0.85 — это важный бизнес-порог, при котором система «понимает», что пользователю нужна помощь.
- **Динамика:** Схема демонстрирует переход системы из состояния «Хаос/Перегруз» в состояние «Контроль» за один цикл обмена сообщениями.

4. Диаграмма последовательности: Добавление задачи через ИИ (NLP)

Эта схема показывает магию Flowra: как обычная фраза «Напомни сделать реферат в пятницу на 3 часа» превращается в структурированную задачу.



Данная диаграмма визуализирует процесс обработки естественного языка (Natural Language Processing) при создании новой задачи (прецедент ВИ-02). Она демонстрирует взаимодействие между пользователем, интерфейсом приложения, внутренним бэкендом и внешней нейросетью.

1. Участники процесса (Participants)

- **Пользователь (User):** вводит запрос в свободной форме (голосом или текстом).
- **Фронтенд (App):** мобильное приложение, которое служит оболочкой для ввода и отображения результатов распознавания.
- **Бэкенд (API):** сервер Flowra, который выступает посредником («мостом») между приложением и искусственным интеллектом.
- **AI Service (OpenAI):** внешняя нейросеть, выполняющая самую сложную часть работы — анализ смысла текста.
- **База данных (DB):** конечное хранилище, где фиксируется уже подтвержденная задача.

2. Пошаговый разбор сценария (Messages)

1. **Ввод данных:** Пользователь вводит неструктурированную фразу, например: *«Реферат в пятницу на 3 часа»*. Приложение отправляет этот текст на сервер запросом POST /api/tasks/ai-parse.
2. **Обращение к ИИ:** Бэкенд перенаправляет этот текст во внешнюю нейросеть OpenAI. В запросе указывается инструкция провести **NER (Named Entity Recognition)** — извлечение именованных сущностей.
3. **Распознавание (AI Response):** Нейросеть анализирует текст и возвращает структурированный ответ в формате **JSON**. Она сама понимает, что:
    - *«Реферат»* — это заголовок (title).
    - *«3 часа»* — это длительность в минутах (180).
    - *«В пятницу»* — это крайний срок (deadline).
4. **Предпросмотр:** Бэкенд передает эти данные в мобильное приложение. Фронтенд рисует пользователю «предзаполненную карточку». Это

критический шаг: система не сохраняет задачу молча, а дает пользователю проверить, правильно ли его понял робот.

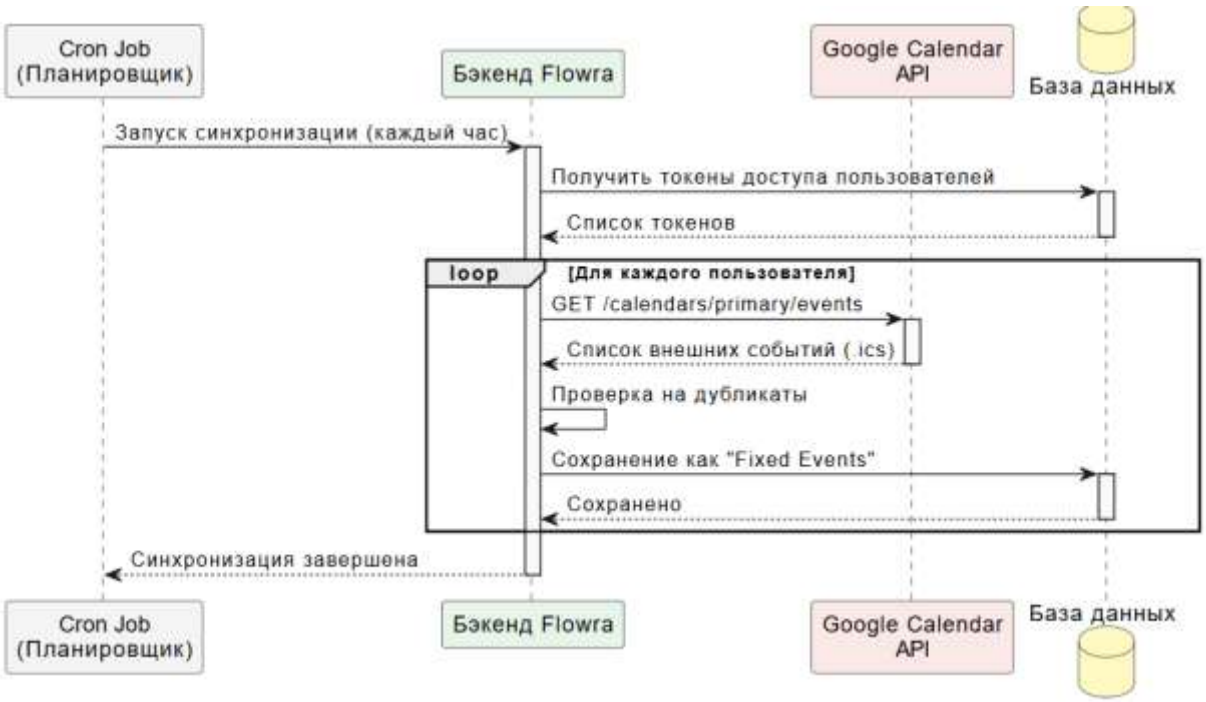
5. **Подтверждение:** Пользователь проверяет данные и нажимает кнопку «Подтвердить». Приложение отправляет финальный запрос POST /api/tasks/confirm.
6. **Финальное сохранение:** Бэкенд записывает задачу в базу данных PostgreSQL. После получения подтверждения от БД сервер сообщает приложению об успехе.
7. **Завершение:** Мобильное приложение обновляет список задач на экране и уведомляет пользователя об успешном добавлении.

3. Технические особенности схемы

- **Интеграция с внешним API:** Схема наглядно показывает, что Flowra не тратит ресурсы смартфона на сложные вычисления ИИ, а использует облачные мощности через Бэкенд.
- **Двухэтапный процесс:** Четко видно разделение на «Парсинг» (распознавание) и «Коммит» (сохранение). Это защищает базу данных от записи случайного или неверно распознанного мусора.
- **Формат обмена:** Основным языком общения между компонентами здесь выступает JSON, что делает систему гибкой и современной.

5. Диаграмма последовательности: Синхронизация с Google Календарём

Пользователю важно, чтобы его учебное расписание из внешних календарей автоматически блокировало время во Flowra.



Подробное описание диаграммы последовательности: «Синхронизация с Google Календарем»

Данная диаграмма визуализирует процесс автоматического импорта внешних событий (прецедент ВИ-14). Она демонстрирует работу системы в фоновом режиме, обеспечивая актуальность «железных» (фиксированных) событий в расписании пользователя.

1. Участники процесса (Participants)

- **Cron Job (Планировщик):** специальный системный таймер, который запускает процесс синхронизации по расписанию (например, раз в час).
- **Бэкенд Flowra (API):** основной сервер, выполняющий логику авторизации и обработки данных.
- **Google Calendar API:** внешний программный интерфейс компании Google, предоставляющий данные о событиях пользователя.
- **База данных (DB):** хранилище, где лежат ключи доступа (токены) и куда записываются импортированные события.

2. Пошаговый разбор сценария (Messages)

1. **Триггер:** Процесс начинается не по нажатию кнопки пользователем, а по команде от **Cron Job**. Это делает синхронизацию незаметной и удобной.
2. **Получение доступов:** Бэкенд обращается к базе данных, чтобы получить список токенов доступа (OAuth2 tokens). Эти токены позволяют серверу Flowra легально «спрашивать» Google о расписании конкретных пользователей.
3. **Цикл обработки (Блок loop):** На схеме выделен прямоугольник loop. Это означает, что все действия внутри него повторяются для каждого пользователя, привязавшего свой Google-аккаунт.
4. **Запрос к Google:** Бэкенд отправляет запрос GET к серверам Google. В ответ он получает список событий в формате, близком к .ics (название занятия, время начала и конца).
5. **Интеллектуальная фильтрация (Self-Message):** Бэкенд выполняет важную внутреннюю проверку — **«Проверка на дубликаты»**. Это нужно для того, чтобы одно и то же занятие не добавилось в расписание дважды при каждой синхронизации.
6. **Сохранение (Fixed Events):** Новые найденные события записываются в базу данных как **«Fixed Events»**.
    - *Важно:* Для алгоритма Flowra это означает, что данные слоты времени теперь «заблокированы», и при автогенерации плана задачи туда ставиться не будут.
7. **Завершение:** После того как цикл прошел по всем пользователям, Бэкенд сообщает планировщику, что работа успешно завершена.

3. Технические особенности схемы

- **Автономность:** Это единственная диаграмма, где нет актора «Пользователь». Это подчеркивает автоматизацию системы Flowra.
- **Блок Loop:** Использование стандартной нотации UML для циклов (loop) показывает, что система масштабируема и рассчитана на работу с множеством пользователей одновременно.

- **Интеграция:** Схема наглядно иллюстрирует концепцию **API-интеграции**, где одно веб-приложение бесшовно обменивается данными с другим.

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ (Лаб. №4)

1. Для чего создается диаграмма последовательности?

Она используется для моделирования логики взаимодействия объектов Системы строго во времени (в хронологическом порядке) в рамках выполнения одного конкретного сценария (прецедента).

2. Скольким вариантам использования соответствует диаграмма последовательности?

Обычно одна диаграмма последовательности соответствует одному конкретному варианту использования (Use Case) или одному его сценарию (например, основному или альтернативному).

3. Что может выступать в качестве объектов на диаграмме последовательности?

В качестве объектов могут выступать экземпляры классов, физические компоненты программного обеспечения (Бэкенд, БД), внешние системы, а также акторы (пользователи), инициирующие взаимодействие.

4. Для чего создается диаграмма компонентов?

Для визуализации физической архитектуры системы, её разбиения на программные модули (библиотеки, пакеты), а также для отображения интерфейсов и зависимостей между этими модулями.

5. С помощью каких элементов на диаграмме можно показать классы объектов?

На диаграмме последовательности классы объектов показываются в виде прямоугольников в верхней части схемы. Внутри прямоугольника указывается имя класса.

6. Как на диаграмме можно показать экземпляры классов объектов?

Имя экземпляра пишется перед двоеточием, за которым следует имя класса, и вся строка подчеркивается. Например: <u>myTask : Task</u>.

7. Что такое порт и для чего он нужен?
Порт — это специфическое «окно» на границе компонента (изображается в виде маленького квадрата). Он нужен для того, чтобы компонент мог принимать и отправлять сообщения через строго определенные интерфейсы, не раскрывая свою внутреннюю структуру.

8. Что такое линия жизни объекта и что она обозначает?
Это вертикальная пунктирная линия под объектом. Она обозначает период времени, в течение которого данный объект существует в системе и может участвовать во взаимодействии.

9. Как могут быть созданы объекты?
Объекты могут создаваться динамически в процессе работы. Это показывается стрелкой с сообщением <<create>>, направленной к прямоугольнику нового объекта, который рисуется на уровне момента его создания (не в самом верху схемы).

10. Каким образом располагаются объекты в схеме и почему?
Объекты располагаются горизонтально слева направо. Крайним слева обычно ставится инициатор (Актор), а далее — остальные объекты по мере того, как они вовлекаются в процесс передачи сообщений.

11. Что такое коннектор и какими виды связей предполагаются к использованию в коннекторах?
Коннектор — это линия, соединяющая два порта или компонента. В коннекторах могут использоваться обычные зависимости (пунктирная стрелка) или связи через интерфейсы (нотация «шарик-гнездо»).

12. Что такое временная шкала, для чего она нужна и как она располагается на схеме?
Временная шкала на диаграмме последовательности подразумевается вертикальной и направлена сверху вниз. Она нужна для отображения хронологического порядка отправки и получения сообщений между компонентами системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №4 были изучены методики проектирования физической архитектуры и динамического поведения программных систем с использованием языка UML.

Применительно к проекту **Flowra** были решены следующие задачи:

1. Разработана **диаграмма компонентов**, которая позволила структурировать приложение в виде микросервисной архитектуры. Система была разделена на клиентский слой (Flutter и Telegram), серверный слой (FastAPI) и слой данных (PostgreSQL, Redis). Это обеспечивает масштабируемость и независимость модулей.
2. Спроектирована серия **диаграмм последовательности**, детально описывающих ключевые бизнес-процессы:
    - Алгоритм автоматической генерации расписания с участием математического оптимизатора.
    - Работа интеллектуального Rescue Mode («Режима спасения») при критической нагрузке.
    - Процесс NLP-парсинга задач на естественном языке с интеграцией внешнего сервиса ИИ.
    - Фоновая синхронизация событий с Google Calendar для блокировки временных слотов.

Полученные модели позволяют перейти от концептуального проектирования к этапу реализации программного кода, имея четкое представление о протоколах взаимодействия между всеми частями системы Flowra. Цели лабораторной работы достигнуты в полном объеме.